

Microservices Fault Tolerance & Circuit Breakers

The Definitive Architectural Master Class for Resilience4j & OpenFeign

■ MASTER CLASS OBJECTIVE:

This architectural guide covers every potential question about Circuit Breakers. It details the 3 internal states, the math behind 5-request trial runs, compares Feign-level vs. Method-level configurations, and provides step-by-step code walks to ensure you can confidently handle any system design question.

Chapter 1: The Three Internal States of a Circuit Breaker

A Circuit Breaker operates exactly like an electrical fuse, protecting healthy microservices from cascading failures:

■ 1. CLOSED (Normal State):

- All network traffic is allowed through to the target downstream microservice.
- Resilience4j tracks request outcomes inside a sliding window. The service is considered healthy.

■ 2. OPEN (Broken State):

- Triggered when the failure rate threshold is breached.
- All incoming traffic is immediately blocked (fails fast) and redirected to the **Fallback Class**. No network calls are sent to the failing service, giving it a cooldown period (e.g., 10 seconds) to recover.

■ 3. HALF-OPEN (Trial State):

- Triggered automatically once the cooldown period expires.
- Allows a configured number of trial requests (e.g. **5 requests**) to test if the service has recovered. If the trial succeeds, the circuit goes back to **CLOSED**. If it fails, it trips back to **OPEN**.

Chapter 2: Types of Circuit Breaker Sliding Windows

Resilience4j measures health using one of two **sliding window types**:

• Count-Based Sliding Window (What your project uses):

Analyzes a fixed number of requests (e.g., the last `10` requests). If 50% fail, it opens. Ideal for high-traffic microservices.

• Time-Based Sliding Window:

Analyzes requests made within a fixed time window (e.g., all requests inside the last `30 seconds`). Ideal for low-traffic services.

Chapter 3: Feign-Level vs. ServiceImpl-Level Circuit Breakers

Feature	Feign-Level (We Used)	ServiceImpl-Level (Alternative)
Separation of Concerns	Perfect. Business logic in `ServiceImpl` does not know fallback handling exists.	Poor. Falling back methods and error handling are mixed into business logic.
What it Protects	Only external HTTP network calls mapped via OpenFeign clients.	Any method (database queries, local CPU-heavy file writing, etc.).
Complexity Chain	Simple and automatic fallback binding via properties.	Advanced. Allows stacking annotations: `@CircuitBreaker`, `@Retry`, `@Bulkhead`.

Chapter 4: Detailed Breakdown of Your 5-Line Configuration

Here are the exact 5 properties we configured in your `application.properties` and the precise math behind them:

1. `spring.cloud.openfeign.circuitbreaker.enabled=true`
-> Globally activates Resilience4j interceptors for all OpenFeign REST client interfaces.
2. `slidingWindowSize=10`
-> Tracks the health outcomes of the last 10 requests.
3. `failureRateThreshold=50`
-> Trips to OPEN if 50% or more requests fail inside the sliding window (e.g. 5 out of 10 fail).
4. `waitDurationInOpenState=10000`
-> Keeps the circuit in the OPEN state for 10 seconds (cooldown) before trying to recover.
5. `permittedNumberOfCallsInHalfOpenState=5`
-> Allows 5 trial calls in the HALF-OPEN state. At a 50% threshold, at least 3 out of 5 trial calls must succeed to CLOSE the circuit. If 3 or more fail, the circuit trips back to OPEN.

Chapter 5: Code walk of Your Feign Client Fallback

Here is the precise architectural flow of how your Feign clients are configured in your code. When a REST call to USER-SERVICE fails, the system automatically redirects the call to your fallback class:

```
@FeignClient(name = "USER-SERVICE", fallback = UserClientFallback.class)
public interface UserClient {
    @GetMapping("/api/users/{id}")
    UserDTO getUserById(@PathVariable("id") Long id);
}

@Component
public class UserClientFallback implements UserClient {
    @Override
    public UserDTO getUserById(Long id) {
        UserDTO fallbackUser = new UserDTO();
        fallbackUser.setName("Fallback User");
        fallbackUser.setRole(Role.TOURIST);
        return fallbackUser; // Returns safe defaults instead of throwing Exception!
    }
}
```